TREE WALKING NOTES (pre / in / post)

There are only THREE DFS orders. Recursion vs stacks is HOW you walk, not a new kind of tree.

```
pre  = me, left, right
in   = left, me, right
post = left, right, me
```

A node has TWO children. Recursion parks one on Python's hidden stack. Iterative = you hold that stack yourself.

1. RECURSION — ONE FUNCTION, MOVE THE PRINT

All 3 are this. Not 3 techniques.

```
def walk(node):
    if node is None:
        return
    # PRE:  print here          me, then kids
    walk(node.left)
    # IN:   print here          left done, me, then right
    walk(node.right)
    # POST: print here          both kids done, then me
```

Same shape for "do something else instead of print": add node.data (sum of non-leaves — skip leaves) track min (smallest_value) swap left/right (mirror, do it BEFORE walking kids)

If the question allows recursion, do this. Do not invent stacks.

2. ITERATIVE — WHAT YOU ACTUALLY NEED

Pre and in do NOT need 2 stacks. One is enough.

```
               recursion     1 stack              2 stacks
pre            print first   YES (Q3 tutorial)    no
in             print middle  YES (A2 Q1)          no
post           print last    S1 (lab)             S2 (A2 Q5)
```

Post is the annoying one: you cannot print me until BOTH kids are done. That is why it has extra versions. You do NOT need all of them.

LEARN THESE 4, ignore the rest:

```
1) recursive walk          (section 1)
2) pre, 1 stack            pop, print, push RIGHT then LEFT
3) in,  1 stack            park me, go left, pop+print, go right
4) post S2, 2 stacks       walk onto stack2, empty stack2 to print
                           (same as S1 reverse, stack2 IS the reverse)
```

PRE vs POST S2 ARE THE SAME LOOP Not like infix/postfix (one machine, flip direction). Iterative in-order is a different shape. Pre and post-S2 ARE cousins:

```
    Pre:     pop -> PRINT              -> push kids
    Post S2: pop -> PUSH onto stack2   -> push kids
             then empty stack2 to print (that reverse IS post-order)

Only extra difference: kid push order is flipped.

    Pre wants left next  -> push RIGHT, then LEFT
    Post S2 wants the walk "me, right, left" on stack2
                         -> push LEFT, then RIGHT
                         emptying stack2 flips it to left, right, me

S1 reverse is the same as S2: a list + .reverse() instead of stack2.
```

Post + 1 stack extra (only if they BAN a second stack / reverse):

```
S1 reverse      walk me-right-left into a list, then list.reverse()
                = lab Solution 1. LEARN this if 1-stack post is asked.
S1 visit-twice  "have I done my right yet?" HARD. lab Solution 2.
                You reinvented this as stackMid/stackRight. PARK IT.
```

3. WHERE YOUR CODE IS

Several files have the SAME algo. Star = the one to trust / submit.

RECURSIVE pre / in / post (print in different slots) * DSA/BT and BST Programs/Binary Tree/01_pre_in_post_order.py DSA/BT and BST Programs/Binary Tree/aryan_try.py (preorder, inorder, postorder — same thing)

RECURSIVE but not a traversal question (same walk, different job) * DSA/Assigent 2/sumOfNonLeafNodes_Template.py sumOfNonLeafNodes — A2 Q3. Submit this. DSA/BT and BST Programs/Binary Tree/13_sum_non_leaf_nodes.py clean copy of Q3 idea DSA/Binary Tree Lab/smallest_value_Template.py DSA/Binary Tree Lab/printSmallerValues_Template.py DSA/Binary Tree Lab/mirrorTree_Template.py DSA/Binary Tree Lab/hasGreatGrandchild_Template.py DSA/Binary Tree Lab/identical_Template.py

PRE iterative — 1 stack * DSA/Binary Tree Tutorial/Q3_Template.py preOrderIterative DSA/BT and BST Programs/Binary Tree/05_pre_order_iterative.py DSA/BT and BST Programs/Binary Tree/aryan_try.py preorderWithStack

IN iterative — 1 stack (A2 Q1) * DSA/Assigent 2/inorderIterative_Template.py inorderIterative <-- SUBMIT this inorderInterative_easeir <-- empty, ignore DSA/BT and BST Programs/Binary Tree/11_inorder_iterative.py clean copy of the same loop

POST S1 — 1 stack (lab) * DSA/Binary Tree Lab/postOrderIterativeS1_Template.py class Solution = reverse trick. LEARN / submit for 1-stack. class Solution02 = visit-twice hard. extra, ignore unless banned reverse DSA/Binary Tree Lab/stack_instead_of_recursion.txt explains WHY reverse works

POST S2 — 2 stacks (A2 Q5) * DSA/Assigent 2/postOrderIterativeS2_Template.py postOrderInterativeS2_correct <-- THIS is S2. Rename to postOrderIterativeS2 before submit postOrderIterativeS2 <-- your visit-twice version. works, but NOT what they named S2 DSA/BT and BST Programs/Binary Tree/12_postorder_iterative.py clean copy of S2 reverse

LEVEL ORDER — queue, not a stack (not DFS) * DSA/Binary Tree Tutorial/Q2_Template.py level_order_traversal DSA/BT and BST Programs/Binary Tree/04_level_order.py

NOT DONE YET (needs BST, not just walking) DSA/Assigent 2/sumGreaterTree_Template.py A2 Q2 DSA/Binary Search Tree Lab/... insert / isBST / remove

4. WHAT EACH ITERATIVE LOOP IS DOING

PRE (1 stack) — print NOW, then schedule kids. Push RIGHT first, then LEFT, so LEFT pops next (left-to-right).

```
if root is None: return
s = Stack();  push(s, root)
while not is_empty(s):
    cur = pop(s)
    print(cur.data, end=" ")
    if cur.right: push(s, cur.right)
    if cur.left:  push(s, cur.left)
```

IN (1 stack) — cannot print until left is done, so PARK myself.

```
if root is None: return
s = Stack();  current = root
while current or not is_empty(s):
    while current:                # go as left as possible
        push(s, current)
        current = current.left
    current = pop(s)              # pop IS the node, not the int
    print(current.data, end=" ")
    current = current.right       # None is fine (no if needed)
```

POST S2 (2 stacks) — do NOT print on the walk. Stack2 reverses for you. Walk is me, then right, then left (push left then right on stack1). Emptying stack2 flips that into left, right, me.

```
    if root is None: return
    s1, s2 = Stack(), Stack()
    push(s1, root)
    while not is_empty(s1):
        cur = pop(s1)
        push(s2, cur)                  # save, don't print
        if cur.left:  push(s1, cur.left)
        if cur.right: push(s1, cur.right)
    while not is_empty(s2):
        print(pop(s2).data, end=" ")
```

POST S1 reverse (1 stack) — same walk, list instead of stack2.

```
    s = [root];  result = []
    while s:
        cur = s.pop()
        result.append(cur.val)
        if cur.left:  s.append(cur.left)
        if cur.right: s.append(cur.right)
    result.reverse()
    return result
```

Why reverse works: walk order = me, right, left reverse = left, right, me = POST

5. WHY YOU GOT STUCK

You tried to do IN-ORDER with the POST 2-stack reverse.

The reverse trick is POST-only: me, right, left --(reverse)--> left, right, me

In-order is left, me, right. Reverse of that is right, me, left, which is NOT something stack1 naturally walks. So 2 stacks do not help in-order.

pop() returns the BSTNode you pushed, not the integer. StackNode.data = the tree node BSTNode.data = the number print(node.data); current = node.right is legal

while is_empty(s) never runs after you just pushed. You want while not is_empty(s)

6. EXAM PICKER

```
"recursive" / no stack mentioned     -> section 1, move the print/add
"iterative" + pre                    -> 1 stack, print then kids
"iterative" + in-order + 1 stack     -> park, go left, pop print, go right
"post" + 2 stacks                    -> S2 reverse (A2 Q5)
```

```
"post" + 1 stack                    -> S1 reverse (lab Solution)
"level order" / "each level"        -> QUEUE, not stack
```

If they did not ban reverse / a second list, do NOT write visit-twice.

7. A2 STATUS

```
Q1 inorderIterative             DONE  (1 stack in-order)
Q3 sumOfNonLeafNodes            DONE  (recursion, return 0 not None)
Q5 postOrderIterativeS2         LOGIC DONE in postOrderInterativeS2_correct
                                rename to postOrderIterativeS2, point main at it
Q2 sumGreaterTree               WAIT for BST lab
                                (walk right, me, left and accumulate)
```